

DSS5201 Data Visualisation

Week 5 Data Manipulation

David CHEW

Semester 2 AY25/26

Last Week: Importing Data

We learnt how to import data from external sources.

- Plain Text Files
- CSV Files
- Excel Files
- JSON Files
- Data from the Web

We also went through some basic ways to clean and explore data.

This Week: Data Manipulation

This week, we cover *data manipulation* a.k.a *data wrangling* with `pandas`:

- String and `datetime` manipulation
- Column manipulation
- Row manipulation
- Grouping and group summaries

Data Manipulation

What is Data Manipulation/Data Wrangling?

“Data janitor work ...”



Artwork by Allison Horst

What is Data Manipulation/Data Wrangling?

It is extremely rare that data come in the exact format we want.

We often need to do some or all of the following:

- Create new variables
- Compute data summaries
- Rename variables
- Reorder columns/rows to make data easier to work with
- ... and many more

You've already seen some of these in Weeks 3-4. This week, we will go deeper with a real data set:

- Flights that departed New York City in 2013 from the `nycflights13` library.

Prerequisites

Let's first import the necessary libraries into our environment.

```
1 import calendar
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
```

The `flights` data come from the `nycflights13` library:

```
1 from nycflights13 import flights
2 flights.columns
```

```
Index(['year', 'month', 'day', 'dep_time', 'sched_dep_time', 'dep_delay',
      'arr_time', 'sched_arr_time', 'arr_delay', 'carrier', 'flight',
      'tailnum', 'origin', 'dest', 'air_time', 'distance', 'hour', 'minute',
      'time_hour'],
      dtype='str')
```

flights Data

A quick peek ...

1 flights.head()

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight
0	2013	1	1	517.0	515	2.0	830.0	819	11.0	UA	154
1	2013	1	1	533.0	529	4.0	850.0	830	20.0	UA	1714
2	2013	1	1	542.0	540	2.0	923.0	850	33.0	AA	114
3	2013	1	1	544.0	545	-1.0	1004.0	1022	-18.0	B6	725
4	2013	1	1	554.0	600	-6.0	812.0	837	-25.0	DL	461

flights Variables

19 variables on flights to and from different airports in New York City in 2013.

Variable	Description
year, month, day	Year, month, day of departure
dep_time, arr_time	Actual departure, arrival times
dep_delay, arr_delay	Actual departure, arrival delays
sched_dep_time, sched_arr_time	Scheduled departure, arrival times
hour, minute	Hours, minute of scheduled departure
time_hour	Date and time of scheduled departure
carrier	2-letter carrier abbreviation
tailnum	Plane tail number
flight	Flight number
origin, dest	Origin, destination airports
air_time	Amount of time spent in the air
distance	Distance flown

flights Data Types

```
1 flights.dtypes
```



year	int64
month	int64
day	int64
dep_time	float64
sched_dep_time	int64
dep_delay	float64
arr_time	float64
sched_arr_time	int64
arr_delay	float64
carrier	str
flight	int64
tailnum	str
origin	str

String and datetime Manipulation

Strings

Distinct values in string columns:

```
1 flights['carrier'].unique()
```

<StringArray>

```
['UA', 'AA', 'B6', 'DL', 'EV', 'MQ', 'US', 'WN', 'VX', 'FL', 'AS', '9E', 'F9',  
 'HA', 'YV', 'OO']
```

Length: 16, dtype: str

16 unique carriers in total:

```
1 flights['carrier'].nunique()
```

Strings

`.value_counts()` shows both the unique values and their counts.

This is more insightful – not all `carriers` have the same number of flights.

```
1 flights['carrier'].value_counts().head()
```

carrier

UA 58665

B6 54635

EV 54173

DL 48110

AA 32729

Name: count, dtype: int64

Time-Related Variables

Let's focus on departure time-related variables and how they are stored.

- Usually we use `.head()` and `.tail()` to peek at data.
- `.sample()` is handy if we want to check random rows.

```
1 flights[['time_hour', 'month', 'day',  
2         'sched_dep_time', 'dep_time',  
3         'dep_delay']].sample(5)
```

	time_hour	month	day	sched_dep_time	dep_time	dep_delay
280546	2013-08-01T19:00:00Z	8	1	1544	1742.0	118.0
139973	2013-03-05T12:00:00Z	3	5	729	723.0	-6.0
91686	2013-12-10T12:00:00Z	12	10	754	756.0	2.0
242176	2013-06-22T12:00:00Z	6	22	825	823.0	-2.0
332090	2013-09-25T23:00:00Z	9	25	1950	1949.0	-1.0

datetime

`pd.to_datetime()` parses strings to dates and times:

```
1 flights['time_hour'] = pd.to_datetime(flights['time_hour'])
2 flights['time_hour'].dtype
```

`datetime64[us, UTC]`

- `pandas` uses `datetime64` to store time.
- `us` indicates microsecond precision.

datetime

```
1 flights['time_hour'].tail()
```

```
336771    2013-09-30 18:00:00+00:00
```

```
336772    2013-10-01 02:00:00+00:00
```

```
336773    2013-09-30 16:00:00+00:00
```

```
336774    2013-09-30 15:00:00+00:00
```

```
336775    2013-09-30 12:00:00+00:00
```

```
Name: time_hour, dtype: datetime64[us, UTC]
```

- The last few rows of the data are not from December as we might expect.
-  The rows are not in chronological order!

datetime

When we check the rows, we find that they are not in chronological order:

- What is the start and end dates of the `flights` data?

```
1 flights['time_hour'].min(), flights['time_hour'].max()
```

```
(Timestamp('2013-01-01 10:00:00+0000', tz='UTC'),  
Timestamp('2014-01-01 04:00:00+0000', tz='UTC'))
```

- The length of this time series:

```
1 flights['time_hour'].max() - flights['time_hour'].min()
```

```
Timedelta('364 days 18:00:00')
```

datetime

- Using `.unique()` on the `month` variable reveals the values in the order they first appear in the data set.

```
1 flights['month'].unique()
```

```
array([ 1, 10, 11, 12,  2,  3,  4,  5,  6,  7,  8,  9])
```

- To fix this, sort the rows by `year`, `month`, `day`, and `dep_time`.
- After sorting, reset the row index and overwrite `flights`.

```
1 flights = (  
2     flights  
3     .sort_values(['year', 'month', 'day', 'dep_time'])  
4     .reset_index(drop = True)  
5 )  
6 flights['month'].unique()
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

datetime

Now the last rows correctly show Dec 31, 2013.

```
1 flights['time_hour'].tail()
```

```
336771    2013-12-31 12:00:00+00:00
```

```
336772    2013-12-31 13:00:00+00:00
```

```
336773    2013-12-31 21:00:00+00:00
```

```
336774    2013-12-31 11:00:00+00:00
```

```
336775    2013-12-31 13:00:00+00:00
```

```
Name: time_hour, dtype: datetime64[us, UTC]
```

datetime

We can extract time-related properties of `datetime` object with the `.dt` accessor:

- ... date, year, quarter, month, weekday, day, hour, etc.

```
1 flights['date'] = flights['time_hour'].dt.date
2 flights['date'] = pd.to_datetime(flights['date'])
3 flights['date'].head(2)
```

```
0    2013-01-01
```

```
1    2013-01-01
```

```
Name: date, dtype: datetime64[s]
```

Departure Delays at LGA

Which day of the week, on average, has the worst departure delays at the LaGuardia Airport (LGA)?

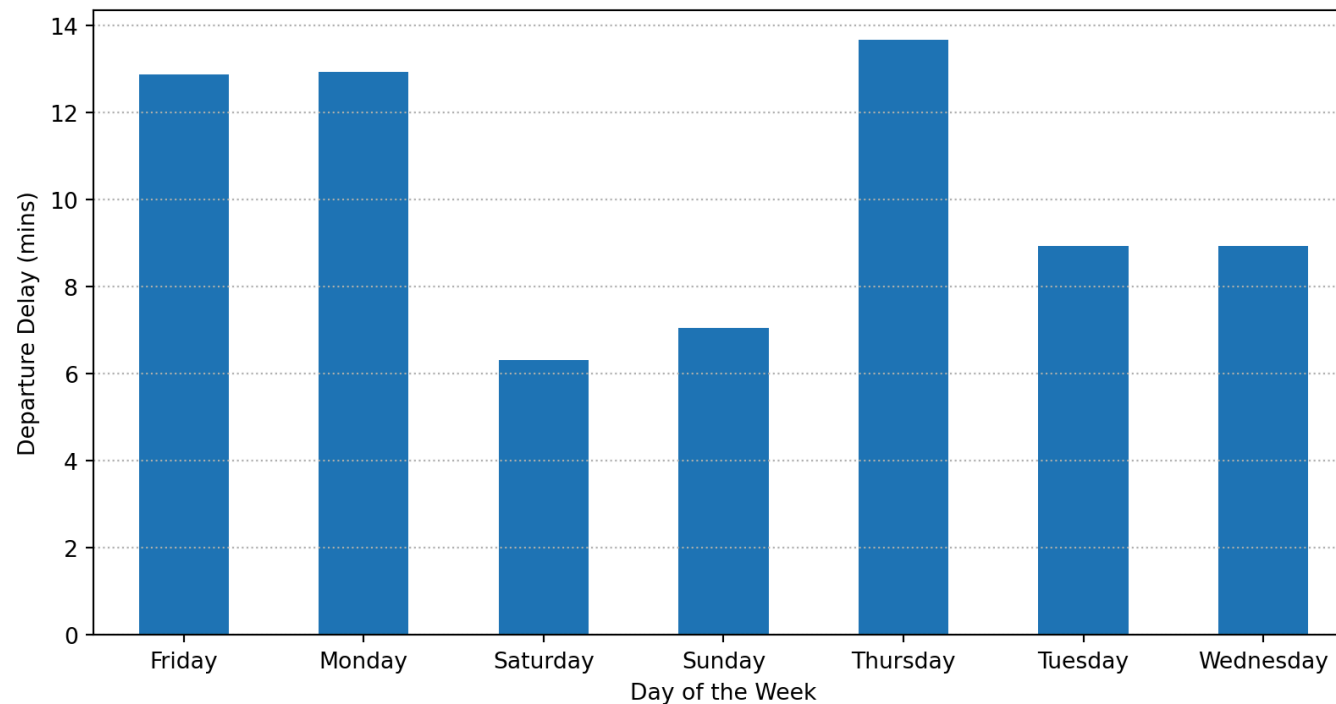
```
1 df = flights[flights['origin'] == 'LGA'].copy()
2 df['weekday'] = df['time_hour'].dt.day_name()
3 df_mean = (
4     df.groupby('weekday')[['dep_delay']]
5     .mean().reset_index()
6 )
7 df_mean
8 # `.copy()` returns an independent copy of the data subset
```

	weekday	dep_delay
0	Friday	12.871005
1	Monday	12.942271
2	Saturday	6.314317
3	Sunday	7.054689
4	Thursday	13.666710
5	Tuesday	8.944221
6	Wednesday	8.934139

Departure Delays at LGA

Bar Plot

```
1 df_mean.plot(kind = 'bar', legend = False, x = 'weekday', y = 'dep_delay',
2             xlabel = 'Day of the Week', ylabel = 'Departure Delay (mins)')
3 plt.xticks(rotation = 0)
4 plt.grid(axis = 'y', linestyle = ':')
5 plt.show()
```



Departure Delays at LGA

Fix weekday Order

In the previous plot, `weekday` is in alphabetical order. This can be misleading.

- We can specify the order they are displayed on the plot.
- Python's `calendar` library provides weekday names in the right order.

```
1 import calendar
2 weekday_order = list(calendar.day_name)
3 weekday_order
```



```
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

- More calendar-related functions: <https://docs.python.org/3/library/calendar.html>

Departure Delays at LGA

Fix weekday Order

Update `weekday` order:

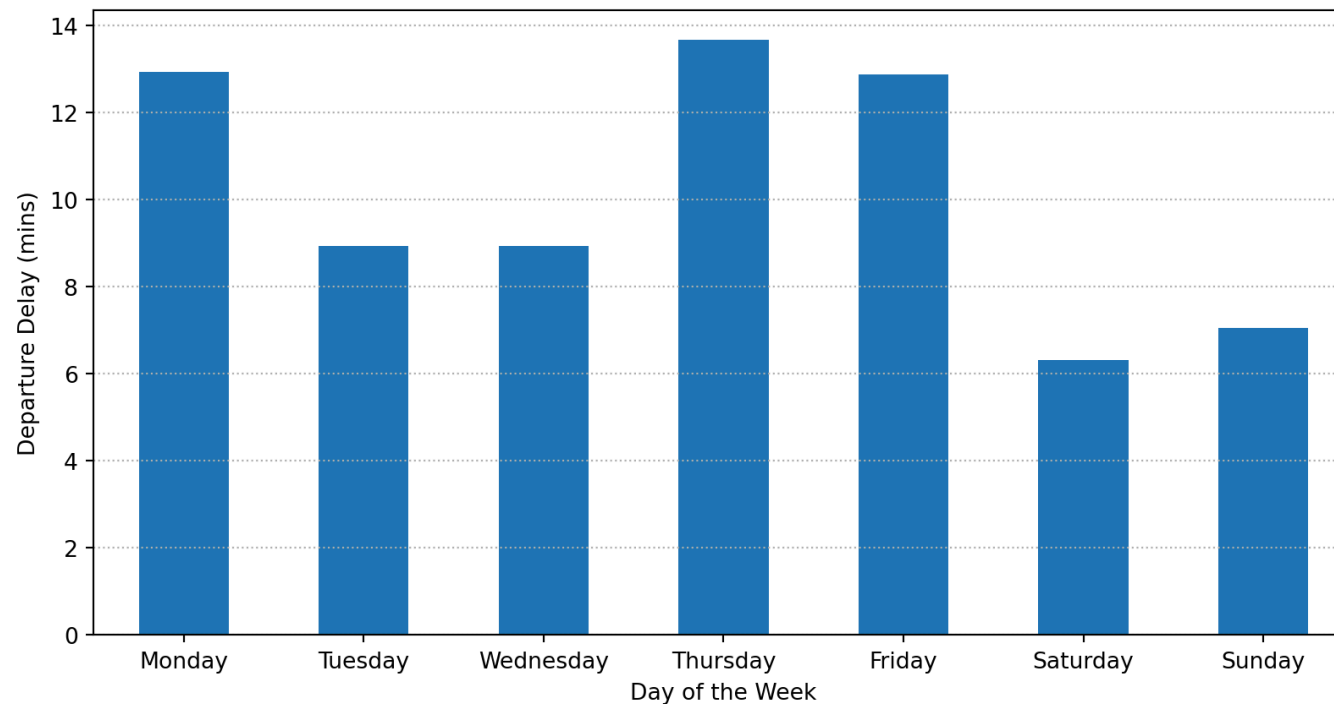
```
1 df_mean['weekday'] = pd.Categorical(df_mean['weekday'],  
2                                   categories = weekday_order,  
3                                   ordered = True)  
4 df_mean = df_mean.sort_values('weekday', ignore_index = True)  
5 df_mean
```

	weekday	dep_delay
0	Monday	12.942271
1	Tuesday	8.944221
2	Wednesday	8.934139
3	Thursday	13.666710
4	Friday	12.871005
5	Saturday	6.314317
6	Sunday	7.054689

Departure Delays at LGA

Revised Bar Plot

```
1 df_mean.plot(kind = 'bar', legend = False, x = 'weekday', y = 'dep_delay',
2               xlabel = 'Day of the Week', ylabel = 'Departure Delay (mins)')
3 plt.xticks(rotation = 0)
4 plt.grid(axis = 'y', linestyle = ':')
5 plt.show()
```



Workflow (so far)

Let's summarise what we've done so far:

1. Subset rows based on certain condition/s.
2. Create or modify columns.
3. Compute group summaries.
4. Fix order for readability.
5. Visualise the results.

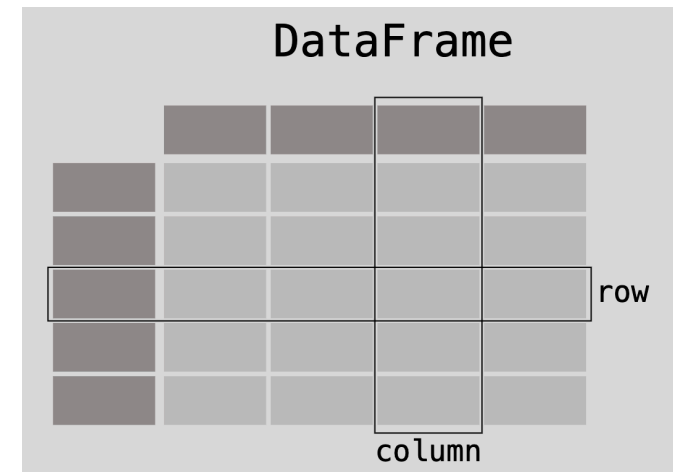
This is a transferable workflow we'll use again and again, not just for the `flights` data.

Column Manipulation

Column Manipulation

We now cover the following **column manipulations** in data frames:

- Subsetting columns
- Adding/removing columns
- Renaming columns
- Re-ordering columns



Subsetting Columns

Let's create a data frame `df` that contains a subset of columns from `flights`.

- When selecting multiple columns, we provide a list of column names inside the brackets:

```
df[['col1', 'col2', 'col3']]
```

- The new data frame keeps the order we specify.

```
1 df = flights[['month', 'day', 'origin', 'dest',  
2               'dep_time', 'sched_dep_time', 'dep_delay']]  
3 df.head()
```

	month	day	origin	dest	dep_time	sched_dep_time	dep_delay
0	1	1	EWR	IAH	517.0	515	2.0
1	1	1	LGA	IAH	533.0	529	4.0
2	1	1	JFK	MIA	542.0	540	2.0
3	1	1	JFK	BQN	544.0	545	-1.0
4	1	1	LGA	ATL	554.0	600	-6.0

Adding Column: dep_status

- Classify flight departure status and store it in a new column, `dep_status`:

```
1 cond = [df['dep_delay'] > 0, df['dep_delay'] <= 0]
2 choices = ['delayed', 'on time or early']
3 df['dep_status'] = np.select(cond, choices, default = 'unknown')
4 df.head()
```

	month	day	origin	dest	dep_time	sched_dep_time	dep_delay	dep_status
0	1	1	EWR	IAH	517.0	515	2.0	delayed
1	1	1	LGA	IAH	533.0	529	4.0	delayed
2	1	1	JFK	MIA	542.0	540	2.0	delayed
3	1	1	JFK	BQN	544.0	545	-1.0	on time or early
4	1	1	LGA	ATL	554.0	600	-6.0	on time or early

The SettingWithCopyWarning

Sometimes when we subset a data frame and then try modify it, the following `warning message` might appear:

SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a DataFrame.

`pandas` is unsure whether we'd like to edit

- A `view` of the original; or
- An independent `copy` as a new object.

Solution: Explicitly use `.copy()` when we plan to modify.

View VS Copy in pandas

When we subset or modify a data frame, `pandas` may return either a `view` or a `copy`.

- `View` shares memory with the original object. Any changes we make to the subset will also affect the original data frame.
- `Copy` is stored independently. Changes will only apply to the new object, without affecting the original.

It raises the `SettingWithCopyWarning` because it does not know which one we'd like to have.

View VS Copy in pandas

View Example

```
1 s1 = pd.Series(['a', 'b', 'c', 'd'])
2 s1_view = s1
3 s1_view[3] = 'CNY'
```

1 s1

0	a
1	b
2	c
3	CNY

dtype: str

1 s1_view

0	a
1	b
2	c
3	CNY

dtype: str

View VS Copy in pandas

Copy Example

```
1 s1 = pd.Series(['a', 'b', 'c', 'd'])
2 s1_copy = s1.copy()
3 s1_copy[3] = 'CNY'
```

1 s1

0	a
1	b
2	c
3	d

dtype: str

1 s1_copy

0	a
1	b
2	c
3	CNY

dtype: str

View VS Copy in pandas

When to use `.copy()`

We don't want to accidentally change our raw data.

- Use `.copy()` when we subset and plan to modify the data.
 - This includes adding/removing/renaming columns, changing values, ...
- If we only summarize or visualize the data, `.copy()` is not required.
 - `.value_counts()`, `.unique()`, `.groupby()`, `.mean()`, `.plot()`, ...

💡 If you are uncertain, it doesn't hurt to use `.copy()`.

Adding Column (Redux): dep_status

We subsetting columns and added a new column `dep_status` previously.

To ensure the original data remains intact, create an independent copy.

```
1 df = flights[['month', 'day', 'origin', 'dest',  
2             'dep_time', 'sched_dep_time', 'dep_delay']].copy()  
3 cond = [df['dep_delay'] > 0, df['dep_delay'] <= 0]  
4 choices = ['delayed', 'on time or early']  
5 df['dep_status'] = np.select(cond, choices, default = 'unknown')  
6 df.head()
```

	month	day	origin	dest	dep_time	sched_dep_time	dep_delay	dep_status
0	1	1	EWR	IAH	517.0	515	2.0	delayed
1	1	1	LGA	IAH	533.0	529	4.0	delayed
2	1	1	JFK	MIA	542.0	540	2.0	delayed
3	1	1	JFK	BQN	544.0	545	-1.0	on time or early
4	1	1	LGA	ATL	554.0	600	-6.0	on time or early

Counting Frequency: dep_status

After that, count the frequency of each status.

- In 2013, 128,432 flights departed late from airports in the New York City.
- There were 8,255 flights with unknown status (flights that were cancelled).

```
1 df['dep_status'].value_counts()
```

```
dep_status
on time or early    200089
delayed             128432
unknown              8255
Name: count, dtype: int64
```

- Can use `normalize = True` for proportions.

Subsetting Columns

By Data Types

Instead of subsetting columns by names, we can also subset by data types.

- The following includes all **integer** columns:

```
1 df.select_dtypes(include = 'int').head()
```

	month	day	sched_dep_time
0	1	1	515
1	1	1	529
2	1	1	540
3	1	1	545
4	1	1	600

- Try `exclude = 'int'` as well.

Subsetting Columns

By Data Types

- The following extracts all **string** columns:

```
1 df.select_dtypes(include = 'object').head()
```



	origin	dest	dep_status
0	EWR	IAH	delayed
1	LGA	IAH	delayed
2	JFK	MIA	delayed
3	JFK	BQN	on time or early
4	LGA	ATL	on time or early

Subsetting Columns

Using `filter()`

`.filter()` is flexible. We can select by names, regular expression (Regex) patterns, or matching substrings.

- To select column by names:

```
1 df.filter(items = ['origin', 'dest']).head(1)
```

	origin	dest
0	EWR	IAH

- Regex string to subset all columns that begins with `dep`.

```
1 df.filter(regex = '^dep').head(1)
```

	dep_time	dep_delay	dep_status
0	517.0	2.0	delayed

Subsetting Columns

Using `filter()`

- To subset all columns that ends with `time`:

```
1 df.filter(regex = 'time$').head(1)
```

	dep_time	sched_dep_time
0	517.0	515

- All columns that contains `dep` anywhere in the name:

```
1 df.filter(like = 'dep').head(1)
```

	dep_time	sched_dep_time	dep_delay	dep_status
0	517.0	515	2.0	delayed

Re-ordering Columns

Manual

To arrange the order of columns, one way is to manually pass a reordered list of column names.

- Simple, but not quite scalable when there are many columns.

```
1 df[['origin', 'dep_time', 'sched_dep_time', 'dep_delay']].head()
```



	origin	dep_time	sched_dep_time	dep_delay
0	EWR	517.0	515	2.0
1	LGA	533.0	529	4.0
2	JFK	542.0	540	2.0
3	JFK	544.0	545	-1.0
4	LGA	554.0	600	-6.0

Re-ordering Columns

Using `sort_index()`

We can sort columns in alphabetical order.

- Can be helpful when we have a large data frame with many columns.

```
1 df.sort_index(axis = 1).head(2)
```

	day	dep_delay	dep_status	dep_time	dest	month	origin	sched_dep_time
0	1	2.0	delayed	517.0	IAH	1	EWR	515
1	1	4.0	delayed	533.0	IAH	1	LGA	529

- For reverse alphabetical order, just specify `ascending = False`.

Re-ordering Columns

Using `sorted()`

- Alternatively, use the built-in Python function `sorted()`:

```
1 df[sorted(df.columns)].head(2)
```

	day	dep_delay	dep_status	dep_time	dest	month	origin	sched_dep_time
0	1	2.0	delayed	517.0	IAH	1	EWR	515
1	1	4.0	delayed	533.0	IAH	1	LGA	529

- To reverse the order:

```
1 df[sorted(df.columns, reverse = True)].head(2)
```

	sched_dep_time	origin	month	dest	dep_time	dep_status	dep_delay	day
0	515	EWR	1	IAH	517.0	delayed	2.0	1
1	529	LGA	1	IAH	533.0	delayed	4.0	1

Re-ordering Columns

Using List Comprehension

`pandas` does not have built-in functions to move specific column(s).

- **Trick:** List comprehension to move `dest` to the front and `origin` to the end.

```
1 df = df[['dest'] + [x for x in df.columns if x != 'dest']]
2 df = df[[x for x in df.columns if x != 'origin'] + ['origin']]
3 df.head(2)
```

	dest	month	day	dep_time	sched_dep_time	dep_delay	dep_status	origin
0	IAH	1	1	517.0	515	2.0	delayed	EWR
1	IAH	1	1	533.0	529	4.0	delayed	LGA

Re-ordering Columns

Using Location Index

Alternatively, we can move columns by a specific location index.

In the following, we move `origin` to position `1` (the second column).

```
1 col = df.pop('origin')
2 df.insert(loc = 1, column = 'origin', value = col)
3 df.head(2)
```

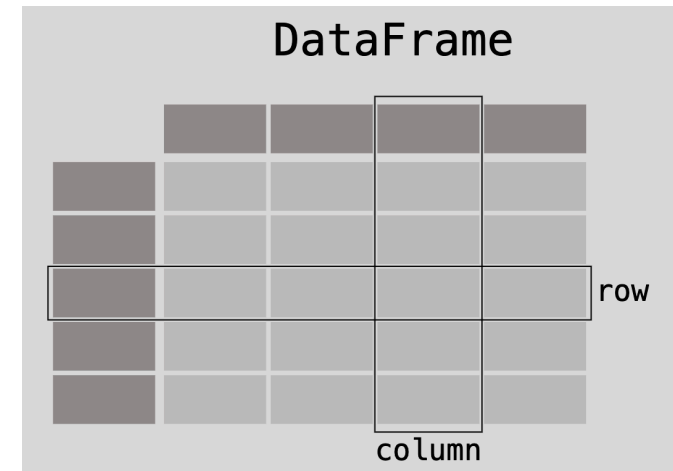
	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
0	IAH	EWR	1	1	517.0	515	2.0	delayed
1	IAH	LGA	1	1	533.0	529	4.0	delayed

Row Manipulation

Row Manipulation

We now look at the following **row manipulations** in data frames:

- Subsetting rows
- Sorting rows by index or by value
- Adding/removing rows



Subsetting Rows

- We subset all flights on January 1st from LGA as `df1`.
- What happened to the indices?

```
1 df1 = df[(flights['month'] == 1) & (flights['day'] == 1) &  
2         (flights['origin'] == 'LGA')].copy()  
3 df1.head()
```

	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
1	IAH	LGA	1	1	533.0	529	4.0	delayed
4	ATL	LGA	1	1	554.0	600	-6.0	on time or early
7	IAD	LGA	1	1	557.0	600	-3.0	on time or early
9	ORD	LGA	1	1	558.0	600	-2.0	on time or early
14	DFW	LGA	1	1	559.0	600	-1.0	on time or early

Subsetting Rows

Resetting Indices

When we subset rows in `pandas`, the resulting data frame preserves the original row indices.

- Not always an issue, but it can be confusing later.
- A good practice is to reset the indices so that it starts from zero.

```
1 df1 = df1.reset_index(drop = True)
2 # `drop = True` prevents the old index from being added as a column
3 df1.head()
```

	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
0	IAH	LGA	1	1	533.0	529	4.0	delayed
1	ATL	LGA	1	1	554.0	600	-6.0	on time or early
2	IAD	LGA	1	1	557.0	600	-3.0	on time or early
3	ORD	LGA	1	1	558.0	600	-2.0	on time or early
4	DFW	LGA	1	1	559.0	600	-1.0	on time or early

Subsetting Rows

Using `isin()`

- We often subset rows based on a list of values:

```
1 df1[(df1['dep_status'] == 'delayed') |  
2      (df1['dep_status'] == 'on time or early')].head()
```

	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
0	IAH	LGA	1	1	533.0	529	4.0	delayed
1	ATL	LGA	1	1	554.0	600	-6.0	on time or early
2	IAD	LGA	1	1	557.0	600	-3.0	on time or early
3	ORD	LGA	1	1	558.0	600	-2.0	on time or early
4	DFW	LGA	1	1	559.0	600	-1.0	on time or early

- Alternatively, we can use `isin()`.

```
1 df1[df1['dep_status'].isin(['delayed', 'on time or early'])]
```

Re-ordering Rows

Using `sort_values()`

Use `.sort_values()` to re-order rows according to values in a particular column.

- By default, rows will be sorted in ascending order.

```
1 df1.sort_values('dep_time', ascending = True).head()
```

	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
0	IAH	LGA	1	1	533.0	529	4.0	delayed
1	ATL	LGA	1	1	554.0	600	-6.0	on time or early
2	IAD	LGA	1	1	557.0	600	-3.0	on time or early
3	ORD	LGA	1	1	558.0	600	-2.0	on time or early
4	DFW	LGA	1	1	559.0	600	-1.0	on time or early

- Try `ascending = False` too.

Re-ordering Rows

By Multiple Columns

- We can also sort by multiple columns.
- Each additional column will be used to break ties in the values of the preceding column.

```
1 df1.sort_values(['sched_dep_time', 'dep_time', 'dep_status'],  
2                 ascending = [True, False, True]).head()
```

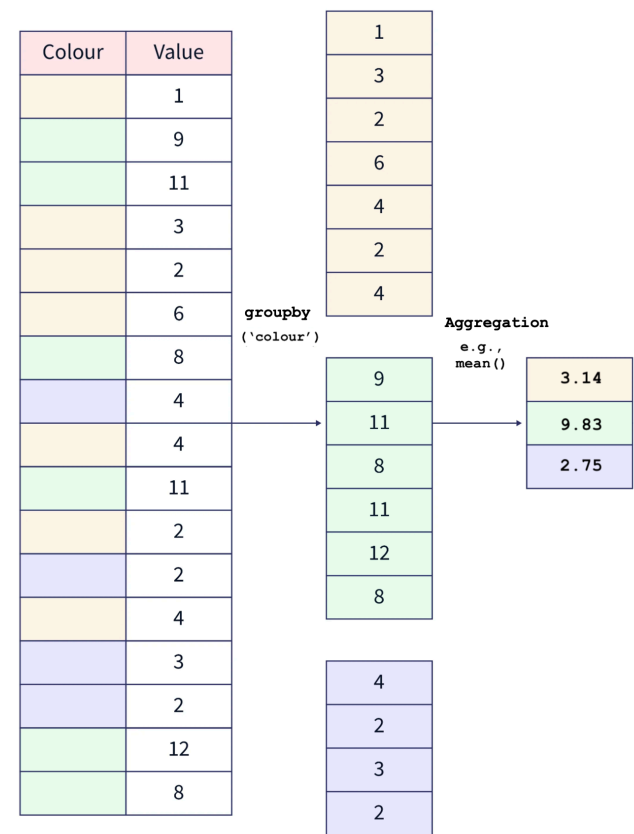
	dest	origin	month	day	dep_time	sched_dep_time	dep_delay	dep_status
0	IAH	LGA	1	1	533.0	529	4.0	delayed
5	FLL	LGA	1	1	600.0	600	0.0	on time or early
6	ATL	LGA	1	1	600.0	600	0.0	on time or early
4	DFW	LGA	1	1	559.0	600	-1.0	on time or early
3	ORD	LGA	1	1	558.0	600	-2.0	on time or early

Grouping and Group Summaries

Grouping and Group Summaries

Now we move from row/column manipulation to group-level analysis.

`pandas` gets even more powerful when we add in the ability to **work with groups**.



We will now work on flights that departed on the New Year's Day
... and select only variables we need.

```
1 df2 = flights[flights['date'] == '2013-01-01']
2 df2 = df2[['month', 'day', 'origin', 'carrier',
3           'dep_time', 'sched_dep_time', 'dep_delay', 'arr_delay']]
4 df2.head()
```

	month	day	origin	carrier	dep_time	sched_dep_time	dep_delay	arr_delay
0	1	1	EWR	UA	517.0	515	2.0	11.0
1	1	1	LGA	UA	533.0	529	4.0	20.0
2	1	1	JFK	AA	542.0	540	2.0	33.0
3	1	1	JFK	B6	544.0	545	-1.0	-18.0
4	1	1	LGA	DL	554.0	600	-6.0	-25.0

Split Data by Groups

The DataFrameGroupBy Object

We first **split the data by groups** using `origin`.

It returns an object that knows how data are split into groups.

```
1 df2.groupby('origin')
```



```
<pandas.api.typing.DataFrameGroupBy object at 0x119d39590>
```

Split Data by Groups

Observations Per Group

- The `.size()` method counts the number of observations in each group.
- It includes rows with missing values.

```
1 df2.groupby('origin').size()
```

origin

EWR 255

JFK 236

LGA 218

dtype: int64

Split Data by Groups

The SeriesGroupBy Object

Next, **select a column**:

- Now Python knows we only care about the `dep_delay` column for each group.
- This produces a `SeriesGroupBy` object.

```
1 df2.groupby('origin')['dep_delay']
```

```
<pandas.api.typing.SeriesGroupBy object at 0x119e1fa10>
```

Group Summaries

Return a Series

When **computing group summaries**, we can return a series or a data frame.

- Single brackets `[]` returns a series:
 - The group labels become the row index.
 - Commonly used for quick analysis on a single column.
 - Here `.count()` returns non-missing observations for each group.

```
1 df2.groupby('origin')['dep_delay'].count()
```

origin

EWR 254

JFK 235

LGA 217

Name: dep_delay, dtype: int64

Group Summaries

Return a Data Frame

- Double brackets `[[[]]]` returns a data frame:
 - This is useful when we need to summarize multiple columns.
 - ... and if we plan to keep the data frame format for merging, exporting or chaining further methods.

```
1 df2.groupby('origin')[['dep_delay']].count()
```

dep_delay	
origin	
EWR	254
JFK	235
LGA	217

Group Summaries

Multiple Columns

We can compute group summaries for multiple columns too:

```
1 df2.groupby('origin')[['dep_delay', 'arr_delay']].mean()
```

	dep_delay	arr_delay
origin		
EWR	16.527559	20.932540
JFK	12.527660	9.217949
LGA	3.548387	8.725581

Group Summaries

Aggregation

Alternatively, we can use `.agg()`, which stands for aggregation.

- `.agg('mean')` is equivalent to `.mean()`.

```
1 df2.groupby('origin')[['dep_delay']].agg('mean')
```



dep_delay	
origin	
EWR	16.527559
JFK	12.527660
LGA	3.548387

Group Summaries

Aggregation Syntax

```
df.groupby('group_var')[['col_to_agg']].agg('agg_function')
```

- Aggregation summarizes each group down to one row.
- The common functions we can pass to `agg()` are the following:

Aggregation function	Description
<code>size()</code>	Number of observations
<code>first(), last()</code>	First, last item
<code>mean(), median()</code>	Mean, median
<code>min(), max()</code>	Min, max
<code>std(), var()</code>	Standard deviation, variance
<code>sum()</code>	Sum of all items
<code>count()</code>	Number of non-missing values

Group Summaries

Multiple Aggregation Functions

We can use more than one aggregation function at a time:

- Here, we calculate the mean and maximum departure delay for each `origin`.
- It returns a data frame with multi-level column index.

```
1 s = df2.groupby('origin')[['dep_delay']].agg(['mean', 'max'])  
2 s
```



	dep_delay	
	mean	max
origin		
EWR	16.527559	379.0
JFK	12.527660	853.0
LGA	3.548387	134.0

Group Summaries

Multiple Aggregation Functions

- For a simpler layout, we can flatten the column index by renaming the columns.
- ... and reset the row index so that `origin` becomes a regular column.

```
1 s1 = df2.groupby('origin')[['dep_delay']].agg(['mean', 'max'])
2 s1.columns = ['mean_dep_delay', 'max_dep_delay']
3 s1 = s1.reset_index()
4 s1
```

	origin	mean_dep_delay	max_dep_delay
0	EWR	16.527559	379.0
1	JFK	12.527660	853.0
2	LGA	3.548387	134.0

- This makes the result easier to read, visualize, and export.

Group Summaries

Modern `.agg()` Syntax

Alternatively, we can define the column, aggregation function, and the new column name directly inside `.agg()`.

- More concise and easier to maintain.

```
1 s1 = df2.groupby('origin').agg(  
2     mean_dep_delay = ('dep_delay', 'mean'),  
3     max_dep_delay = ('dep_delay', 'max')  
4 ).reset_index()  
5 s1
```

	origin	mean_dep_delay	max_dep_delay
0	EWR	16.527559	379.0
1	JFK	12.527660	853.0
2	LGA	3.548387	134.0

Group Summaries

Aggregation by Single Grouping Variable

The following returns the same result.

- Method 1:

```
1 df.groupby('var1')[['col_to_agg']].agg('agg_function')
```

- Method 2 (modern `pandas` ≥ 1.1):

```
1 df.groupby('var1').agg(  
2     new_col = ('col_to_agg', 'agg_function')  
3 )
```

For both methods, we often need to convert the result to a simpler layout for downstream analysis.

Group Summaries

Aggregation by Multiple Grouping Variables

We can also group a data frame by multiple grouping variables.

- Method 1:

```
1 df.groupby(['var1', 'var2'])[['col_to_agg']].agg('agg_function')
```

- Method 2 (modern `pandas` ≥ 1.1):

```
1 df.groupby(['var1', 'var2']).agg(  
2     new_col = ('col_to_agg', 'agg_function')  
3 )
```

Example: Multiple Grouping Variables

An example with Method 2.

The resulting data frame will have a multi-level row index.

```
1 df2.groupby(['carrier', 'origin']).agg(  
2     max_dep_delay = ('dep_delay', 'max'),  
3     mean_arr_delay = ('arr_delay', 'mean')  
4 ).head()
```

		max_dep_delay	mean_arr_delay
carrier	origin		
9E	JFK	255.0	18.294118
AA	EWR	285.0	46.333333
	JFK	131.0	6.166667
	LGA	61.0	10.974359
AS	EWR	-1.0	-14.500000

Example: Multiple Grouping Variables

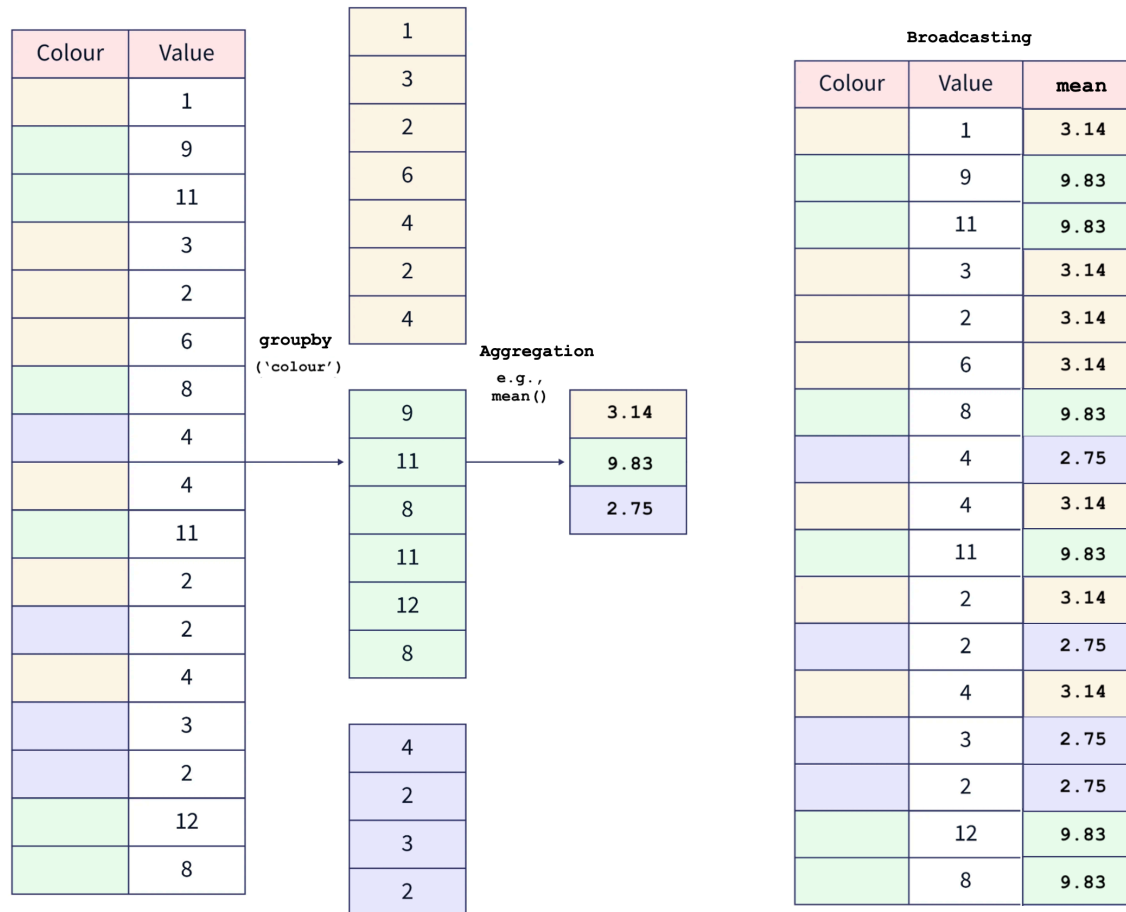
A simpler layout with `.reset_index()`.

- The grouping variables `carrier` and `origin` are in regular columns.
- This format is usually more convenient for downstream analysis.

```
1 df2.groupby(['carrier', 'origin']).agg(  
2     max_dep_delay = ('dep_delay', 'max'),  
3     mean_arr_delay = ('arr_delay', 'mean')  
4 ).reset_index().head()
```

	carrier	origin	max_dep_delay	mean_arr_delay
0	9E	JFK	255.0	18.294118
1	AA	EWR	285.0	46.333333
2	AA	JFK	131.0	6.166667
3	AA	LGA	61.0	10.974359
4	AS	EWR	-1.0	-14.500000

Aggregation and Broadcasting



- **Aggregation** collapses each group in a single row with the specified summaries.
- **Broadcasting** performs the calculation at the group level, then broadcast back the result to the original data frame.

Let's examine flight departure status on Christmas Day by origin airports.

```
1 df3 = flights[(flights['date'] == '2013-12-25')]  
2 df3 = df3[['carrier', 'flight', 'origin', 'sched_dep_time', 'dep_delay']]  
3 df3.head()
```

	carrier	flight	origin	sched_dep_time	dep_delay
329951	B6	745	JFK	2359	10.0
330610	DL	435	JFK	1900	-4.0
330611	DL	407	JFK	1900	-3.0
330612	DL	2391	JFK	1900	-2.0
330615	AA	1029	JFK	1900	0.0

Broadcasting via `.transform()`

- We use `.transform()` after `.groupby()`.
- This creates a new column, `max_delay`, and add it to the end of the existing data frame.
- Each row is assigned the group-level result.

```
1 df3['max_delay'] = df3.groupby('origin')[['dep_delay']].transform('max')
2 df3.head()
```

	carrier	flight	origin	sched_dep_time	dep_delay	max_delay
329951	B6	745	JFK	2359	10.0	234.0
330610	DL	435	JFK	1900	-4.0	234.0
330611	DL	407	JFK	1900	-3.0	234.0
330612	DL	2391	JFK	1900	-2.0	234.0
330615	AA	1029	JFK	1900	0.0	234.0

Aggregation via `.agg()`

- In contrast, `.agg()` collapses each group into a single summary row.

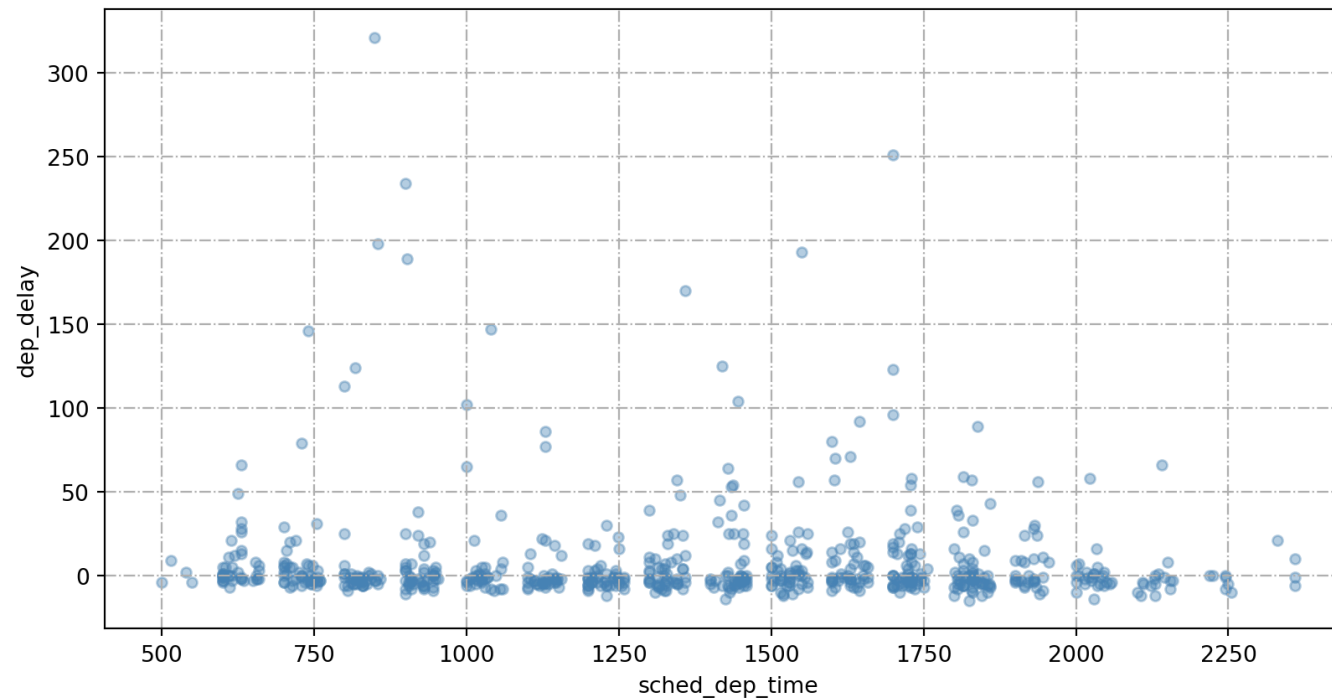
```
1 df3.groupby('origin')[['dep_delay']].agg('max')
```

dep_delay	
origin	
EWR	321.0
JFK	234.0
LGA	251.0

Scatter Plot: Departure Delay on Christmas Day

pandas Plotting

```
1 df3.plot(kind = 'scatter',  
2         x = 'sched_dep_time', y = 'dep_delay',  
3         color = 'steelblue', alpha = 0.4)  
4 plt.grid(axis='both', linestyle = '-.')  
5 plt.show()
```



Scatter Plot: Departure Delay on Christmas Day

matplotlib Plotting

- Recall that `pandas` plotting relies on `matplotlib` under the hood.
- We can produce the same plot with `matplotlib` functions.

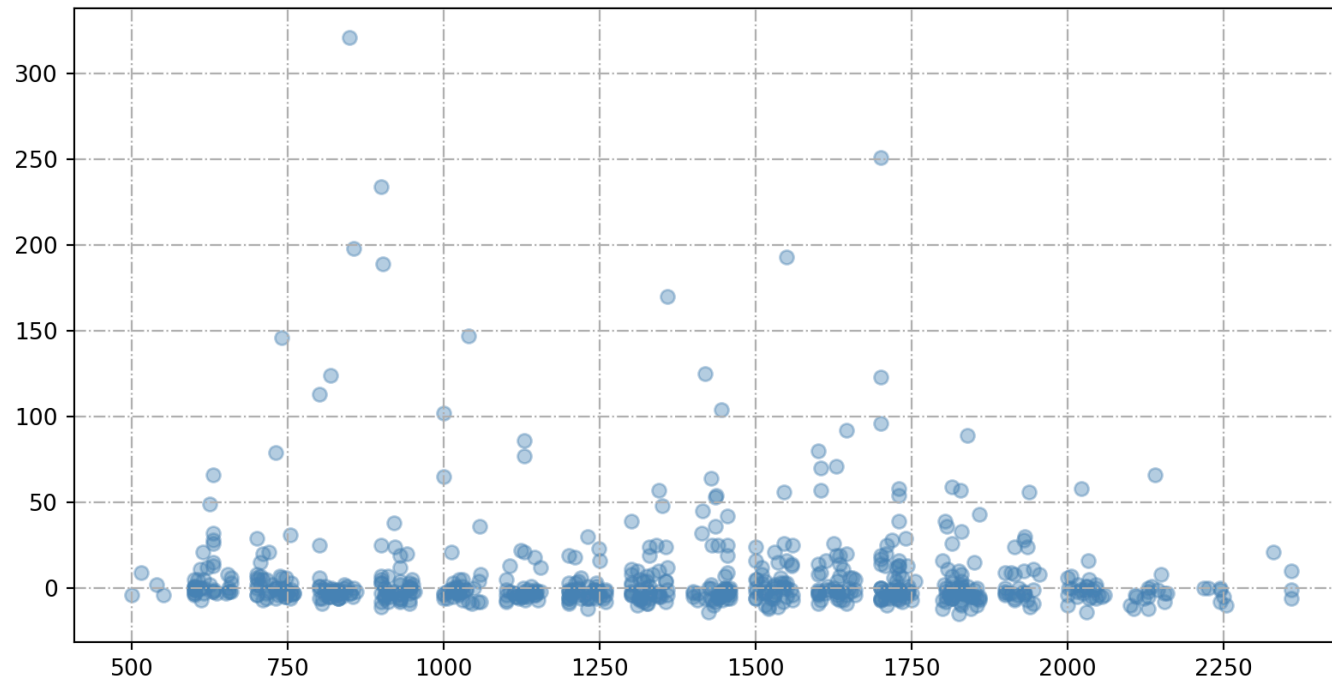
```
1 import matplotlib.pyplot as plt
2 plt.figure(figsize = (10, 5))
3 plt.scatter(df3['sched_dep_time'], df3['dep_delay'],
4             color = 'steelblue', alpha = 0.4)
5 plt.grid(axis='both', linestyle = '-.')
6 plt.show()
```

This is `matplotlib`'s `pyplot` approach, inspired by and based on MATLAB.

- Starts with `plt.scatter()` or similar.
- Ends with `plt.show()` which renders and displays the figure.

Scatter Plot: Departure Delay on Christmas Day

matplotlib Plotting



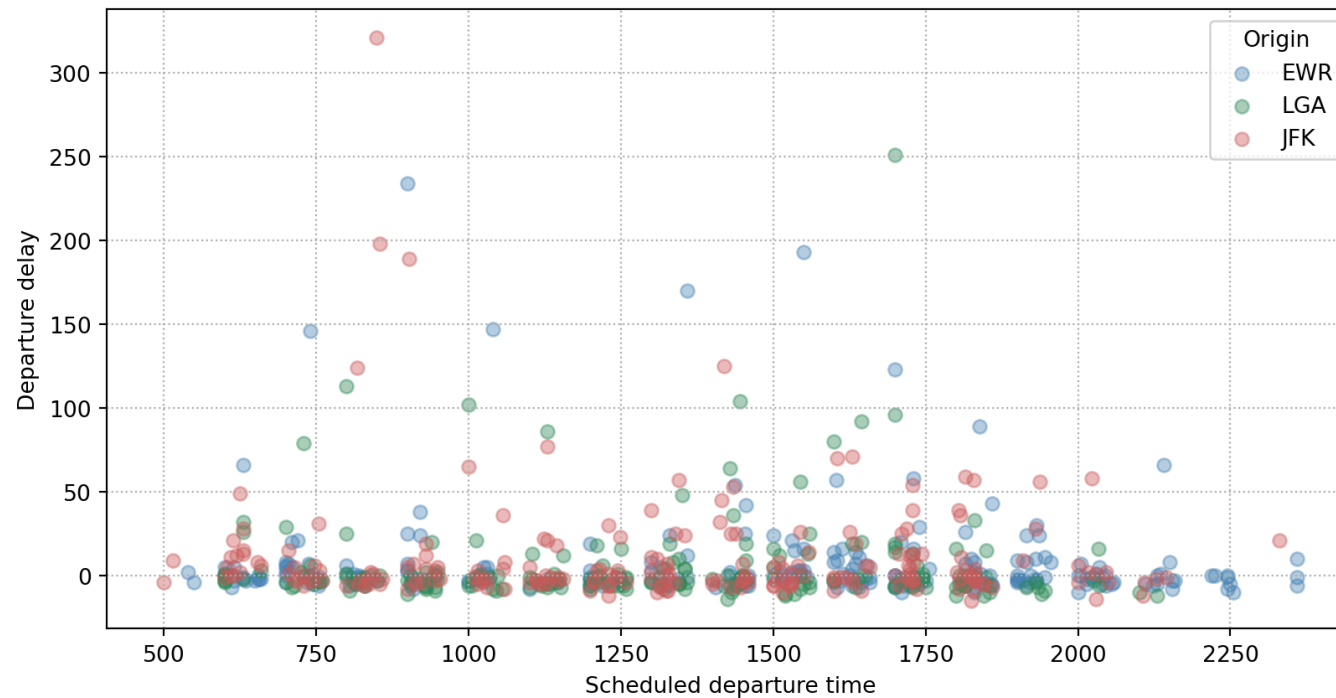
- This is `matplotlib`'s `pyplot` approach.
- In a few weeks, we shall introduce the modern **object-oriented** approach with `matplotlib`.

Scatter Plot: Departure Delay on Christmas Day

By origin

Now we include an additional variable, `origin`, to the plot.

- Map the new variable to the colors of the point.



Scatter Plot: Departure Delay on Christmas Day

By origin

- In the `for` loop, we subset rows for each `origin` and plot those points with pre-defined `colors`.
- This works, but it can become verbose if we have more categories.

```
1 colors = {'EWR': 'indianred',
2           'LGA': 'seagreen',
3           'JFK': 'steelblue'}
4 for x in df3['origin'].unique():
5     subset = df3[df3['origin'] == x]
6     plt.scatter(subset['sched_dep_time'], subset['dep_delay'],
7                 color = colors[x], alpha = 0.4)
8 plt.grid(axis='both', linestyle = ':')
9 plt.legend(colors, title = 'Origin')
10 plt.xlabel('Scheduled departure time')
11 plt.ylabel('Departure delay')
12 plt.show()
```


Scatter Plot: Departure Delay on Christmas Day

Cleaner Code with seaborn

The `seaborn` library provides cleaner and much more readable syntax for grouped scatter plots.

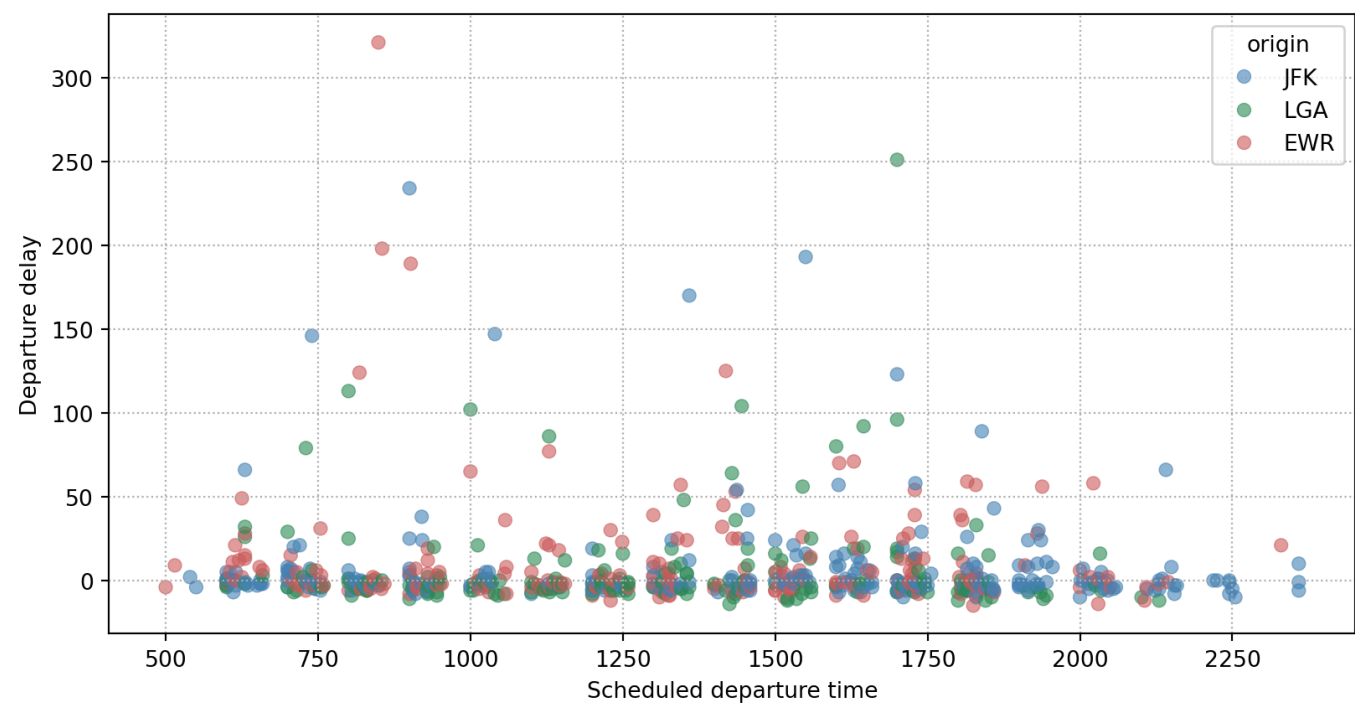
- Directly map `origin` to the `hue` argument. Legends will be automatically added too.
- We shall learn more about it later in the semester.

```
1 import seaborn as sns
2 sns.scatterplot(data = df3,
3                 x = 'sched_dep_time', y = 'dep_delay',
4                 hue = 'origin', palette = colors,
5                 edgecolor = None, alpha = 0.4)
6 plt.grid(axis='both', linestyle = ':')
7 plt.xlabel('Scheduled departure time')
8 plt.ylabel('Departure delay')
9 plt.show()
```



Scatter Plot: Departure Delay on Christmas Day

Cleaner Code with seaborn



Scatter Plot: Departure Delay on Christmas Day

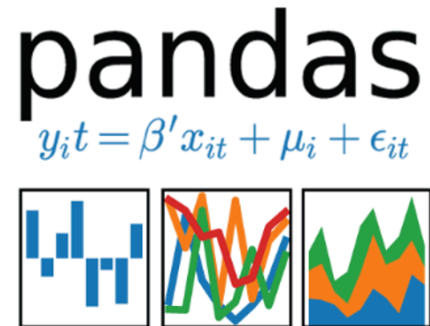
Colour Map

	black		bisque		forestgreen		slategrey
	dimgray		darkorange		limegreen		lightsteelblue
	dimgrey		burlywood		darkgreen		cornflowerblue
	gray		antiquewhite		green		royalblue
	grey		tan		lime		ghostwhite
	darkgray		navajowhite		seagreen		lavender
	darkgrey		blanchedalmond		mediumseagreen		midnightblue
	silver		papayawhip		springgreen		navy
	lightgray		moccasin		mediumspringgreen		darkblue
	lightgrey		orange		mediumaquamarine		mediumblue
	gainsboro		wheat		aquamarine		blue
	whitesmoke		oldlace		turquoise		slateblue
	white		floralwhite		lightseagreen		darkslateblue
	snow		darkgoldenrod		mediumturquoise		mediumslateblue
	rosybrown		goldenrod		azure		mediumpurple
	lightcoral		cornsilk		lightcyan		rebeccapurple
	indianred		gold		paleturquoise		blueviolet
	brown		lemonchiffon		darkslategray		indigo
	firebrick		khaki		darkslategrey		darkorchid
	maroon		palegoldenrod		teal		darkviolet
	darkred		darkkhaki		darkcyan		mediumorchid
	red		ivory		aqua		thistle
	mistyrose		beige		cyan		plum
	salmon		lightyellow		darkturquoise		violet
	tomato		lightgoldenrodyellow		cadetblue		purple
	darksalmon		olive		powderblue		darkmagenta
	coral		yellow		lightblue		fuchsia
	orangered		olivedrab		deepskyblue		magenta
	lightsalmon		yellowgreen		skyblue		orchid
	sienna		darkolivegreen		lightskyblue		mediumvioletred
	seashell		greenyellow		steelblue		deeppink
	chocolate		lawngreen		aliceblue		hotpink
	saddlebrown		honeydew		dodgerblue		lavenderblush
	sandybrown		darkseagreen		lightslategray		palevioletred
	peachpuff		palegreen		lightslategrey		crimson
	peru		lightgreen		slategray		pink
	linen						lightpink

- A list of [named colours](#).
- For colour maps ([cmap](#)), see [here](#).

Scatter Plot: Departure Delay on Christmas Day

matplotlib and seaborn



The `seaborn` syntax will be more comprehensible in a few week's time.

For now, focus on its purpose:

- It creates a scatter plot with scheduled departure time and departure delay.
- ... and also use color (hue) to denote different flight origins.

Compared to `matplotlib`, `seaborn` offers greater flexibility and more concise code.

Summary

This week, we covered *data manipulation* a.k.a *data wrangling* with `pandas`:

- String and `datetime` manipulation
- Column manipulation
- Row manipulation
- Grouping and group summaries

`pandas`' functions are intuitive for data sets that are nicely organized.

In real life, data often arrive in irregular forms... 

- **Next class:** Tidy data